

Monitoramento de Proposições Legislativas — Suinocultura

Área: Dados e Inteligência — ABCS

Responsável técnico: Erick Silva — Analista de BI

Data: 20/08/2026

Versão: 1.0

Status: MVP em desenvolvimento

1. Contexto

A ABCS precisa acompanhar proposições legislativas que afetam o setor de suinocultura. Hoje, o time político faz esse acompanhamento manualmente pelo portal da Câmara dos Deputados: abre a página, busca pelo número do projeto, lê a ementa, verifica o autor, identifica em qual comissão está e anota a data da última movimentação.

Esse processo é repetitivo, sujeito a falhas e consome tempo de profissionais cuja competência é análise política, não coleta de dados.

Este projeto automatiza essa coleta e entrega os dados organizados em dashboards e planilhas atualizados diariamente, permitindo que o time político se concentre em análise e articulação.

2. Problema

O time político precisa responder diariamente:

- Quais proposições relacionadas à suinocultura estão tramitando?
- Quem é o autor e de qual partido/estado?
- Em qual comissão a proposição está?
- Quando foi a última movimentação?
- Existem proposições novas que ainda não estavam no radar?

Hoje, cada uma dessas perguntas exige navegação manual no portal da Câmara. Não há histórico consolidado, não há alertas e não há visão comparativa.

3. Solução proposta

Construir uma pipeline de dados que extrai informações da API de Dados Abertos da Câmara dos Deputados, armazena em banco de dados local e disponibiliza em

formatos consumíveis pelo time político (dashboard Power BI e/ou planilha exportada).

O projeto cobre da engenharia de dados até a análise: extração, transformação, armazenamento, modelagem e visualização.

4. Fonte de dados

A fonte é a API v2 de Dados Abertos da Câmara dos Deputados, mantida pela própria Câmara com acesso público e gratuito.

Endpoints utilizados:

Endpoint	Finalidade
/proposicoes	Listagem e descoberta de proposições (com filtro por tema)
/proposicoes/{id}	Dados completos da proposição (ementa, situação, órgão atual)
/proposicoes/{id}/autores	Deputados autores e coautores
/proposicoes/{id}/temas	Classificação temática da Câmara
/proposicoes/{id}/tramitacoes	Histórico de movimentações
/referencias/proposicoes/codTema	Tabela de referência dos 31 temas legislativos

A API utiliza paginação com limite de 100 registros por página.

5. Estratégia de extração

A API da Câmara classifica cada proposição em temas legislativos (31 no total). O tema 64 — "Agricultura, Pecuária, Pesca e Extrativismo" — concentra as proposições do setor agropecuário. No levantamento realizado em agosto de 2026, esse tema contém aproximadamente 600 proposições.

A extração usa duas camadas de filtragem:

Filtro grosso (na extração): buscar todas as proposições classificadas no tema 64. Isso reduz o universo de ~27.000 proposições para ~600, sem depender de palavras-chave na chamada à API.

Filtro fino (na camada analítica): aplicar a lista de palavras-chave da ABCS (suinocultura, suínos, aves, granjas, rações, sanidade, etc.) sobre a ementa das

proposições já armazenadas. Isso identifica quais das 600 são diretamente relevantes para o setor.

Essa separação permite adicionar novas palavras-chave a qualquer momento sem precisar refazer a extração. A base já contém o universo agropecuário completo; o filtro de relevância é apenas uma consulta interna.

Caso o time político identifique proposições relevantes classificadas em outros temas (ex.: "Meio Ambiente" para maus-tratos, "Trabalho e Emprego" para trabalhador rural), basta incluir o código do tema adicional na extração e rodar uma carga complementar.

6. Arquitetura

6.1. Visão geral

API Câmara dos Deputados

|
| requests (Python)



Scripts Python (extração + transformação)

|
| psycopg2 / SQLAlchemy



PostgreSQL local

|— schema raw → resposta original em JSONB
|— schema staging → dados normalizados em tabelas
|— schema analytics → views prontas para consumo
|
| conexão localhost



Power BI Desktop / Export CSV

6.2. Componentes

Scripts Python: responsáveis pela extração (chamadas à API com paginação), transformação (parsing do JSON, normalização) e carga (inserção/upsert no banco). Executados localmente na máquina do analista.

PostgreSQL local: banco de dados relacional instalado na máquina. Armazena os dados em três schemas com finalidades distintas (detalhados na seção de modelagem).

Task Scheduler (Windows) ou cron (Linux): agenda a execução diária do script de carga incremental. Sem dependência de ferramentas externas de orquestração.

Power BI Desktop: conecta diretamente ao PostgreSQL local via localhost:5432 para visualização. Alternativamente, os dados podem ser exportados como CSV para consumo em planilha.

6.3. Decisão de infraestrutura

Este projeto utiliza infraestrutura local (máquina do analista) e não depende de serviços em nuvem. Essa decisão foi tomada porque:

- O volume de dados é pequeno (centenas, não milhões de registros).
- O consumidor é interno (time político da ABCS, menos de 10 pessoas).
- A independência da infraestrutura existente permite iteração rápida sem dependências de terceiros.
- Caso o projeto prove valor, a migração para ambiente compartilhado pode ser feita posteriormente sem alteração na lógica de negócio.

7. Modelagem de dados

7.1. Schema raw

Preserva a resposta original da API em formato JSONB. Finalidade: auditoria, rastreabilidade e possibilidade de reprocessamento.

Tabela	Conteúdo
--------	----------

raw.proposicoes_json	JSON completo retornado por /proposicoes/{id}
----------------------	---

7.2. Schema staging

Dados normalizados e estruturados, prontos para consultas.

Tabela	Chave	Conteúdo
staging.proposicao	id_proposicao	Dados da proposição: tipo, número, ano, ementa, situação atual, órgão, data de apresentação
staging.proposicao_autor	id_proposicao + id_autor	Relação entre proposição e seus autores (deputados, comissões)
staging.proposicao_tema	id_proposicao + cod_tema	Relação entre proposição e temas legislativos
staging.tramitacao	id_proposicao + sequencia	Histórico de movimentações de cada proposição
staging.dim_tema	cod_tema	Tabela de referência com os 31 temas da Câmara

7.3. Schema analytics

Views que consolidam os dados para consumo direto.

View	Finalidade
analytics.vw_monitoramento	Visão principal: proposição + autor + situação + última tramitação + keywords encontradas + dias sem movimentação
analytics.vw_tramitacoes	Histórico completo de tramitações para drill-down

7.4. Regras de integridade

- **Proposição:** id da API como chave primária. Operação de upsert (insert se novo, update se já existe).
- **Tramitação:** chave composta id_proposicao + sequencia. Estrutura histórica — novos registros são inseridos, nunca sobrescritos.
- **Ano:** preservado conforme a API, sem derivar de dataApresentacao. São atributos independentes.
- **Autores e temas:** relações N:N armazenadas em tabelas próprias.

8. Fluxo de carga

8.1. Carga histórica (execução única)

/proposicoes?codTema=64 (paginação completa)

|



~600 IDs únicos

|

para cada ID:

|— /proposicoes/{id} → detalhes

|— /proposicoes/{id}/autores → autores

|— /proposicoes/{id}/temas → temas

|— /proposicoes/{id}/tramitacoes → histórico

|



INSERT/UPSERT no PostgreSQL

8.2. Carga incremental (execução diária)

Utiliza os parâmetros dataInicio e dataFim para identificar proposições que tiveram movimentação recente.

A janela de busca utiliza sobreposição de segurança (D-3 até o dia atual) para garantir que proposições não sejam perdidas caso uma execução falhe ou atrase.

A sobreposição não gera duplicatas porque todas as operações são idempotentes (upsert por chave).

8.3. Idempotência

Qualquer execução pode ser repetida sem efeitos colaterais. A mesma janela de datas pode ser processada múltiplas vezes: registros novos são inseridos, existentes são atualizados, nenhum é duplicado.

9. Palavras-chave

A lista de palavras-chave é mantida como configuração no banco (tabela ou arquivo de parâmetros). A versão inicial contém 22 termos selecionados pelo time técnico e de mercado da ABCS:

Suinocultura, Suínos, Proteína animal, Rações animais, Sanidade, Animais de produção, Genética suína, Granjas, Integração, Agroindústria, Animais de

interesse econômico, Avicultura, Aves, Frango, Milho, Produtor rural, Trabalhador rural, Deje to de animais, Maus tratos, Rótulo, Gaiolas, Animais domésticos.

O matching é feito por busca textual na ementa da proposição (operação ILIKE no PostgreSQL). Não utiliza processamento de linguagem natural nem modelos de IA nesta fase.

Novas palavras-chave podem ser adicionadas a qualquer momento. A alteração afeta apenas a camada analítica (views), sem necessidade de nova extração.

10. Produto entregue — MVP

10.1. O que o MVP entrega

Uma view atualizada diariamente contendo, para cada proposição relevante:

Informação	Exemplo
Identificação	PL 1234/2025
Ementa	Dispõe sobre normas sanitárias para criação de suínos...
Autor	Dep. Fulano (PP/SC)
Situação atual	Tramitando em regime de prioridade
Órgão atual	CAPADR — Comissão de Agricultura
Última movimentação	Designado relator — 18/08/2026
Dias sem movimentação	2
Palavras-chave encontradas	suínos, sanidade
Link direto	URL para o portal da Câmara

Essa view alimenta um dashboard Power BI ou é exportada como planilha, conforme preferência do time político.

10.2. O que o MVP não inclui

- Pautas de comissões e votações (previsto para fase posterior).
- Dados do Senado Federal.
- Alertas automáticos por e-mail ou mensagem.

- Classificação por inteligência artificial ou NLP.
 - Scoring de relevância ponderado.
-

11. Análise de viabilidade

11.1. Volume de dados

Métrica	Valor estimado
Proposições no tema 64	~600
Autores (estimativa: 1,5 por proposição)	~900 registros
Temas por proposição (média ~2)	~1.200 registros
Tramitações (estimativa: 5 a 15 por proposição)	3.000 a 9.000 registros
Total de registros no banco	< 15.000

Esse volume é trivial para qualquer máquina. Um notebook com 8 GB de RAM e um SSD padrão comporta sem qualquer impacto no desempenho.

11.2. Consumo de disco

Estimativa conservadora para todas as tabelas (incluindo JSONB na RAW): menos de 50 MB. O PostgreSQL em si ocupa cerca de 200 MB instalado. Total: menos de 300 MB.

11.3. Chamadas à API

A carga histórica realizará aproximadamente:

Chamada	Quantidade
Listagem de proposições (6 páginas)	6
Detalhes por proposição	600
Autores por proposição	600
Temas por proposição	600
Tramitações por proposição	600
Total	~2.400 chamadas

Com um intervalo de 0,5 segundo entre chamadas (para não sobrecarregar a API), a carga histórica completa leva aproximadamente 20 minutos.

A carga incremental diária realiza significativamente menos chamadas — apenas as proposições que tiveram movimentação nos últimos 3 dias.

11.4. Custos

Item	Custo
API da Câmara	Gratuita (dados abertos)
PostgreSQL	Gratuito (open source)
Python + bibliotecas	Gratuito (open source)
Power BI Desktop	Gratuito (versão desktop)
Infraestrutura	Máquina já existente
Total	R\$ 0

A publicação do dashboard para acesso web via Power BI Service requer licença Pro (~R\$ 90/mês por usuário), mas isso é decisão futura e não bloqueia o MVP.

11.5. Riscos

Risco	Mitigação
API da Câmara fora do ar	Janela de segurança D-3 recupera dados na execução seguinte
Máquina desligada no horário da carga	Carga idempotente — basta executar manualmente ou aguardar o próximo dia
Mudança na estrutura da API	Schema RAW com JSONB preserva o formato original para diagnóstico
Keywords insuficientes	Adição de novas keywords não requer reprocessamento da extração
Proposição relevante em outro tema	Inclusão do tema adicional com carga complementar pontual

12. Requisitos mínimos

12.1. Hardware

- Máquina com 4 GB de RAM ou mais (recomendado 8 GB).
- 500 MB de espaço livre em disco.

- Conexão com internet (para chamadas à API).

12.2. Software

Software	Versão mínima	Finalidade
Python	3.10+	Scripts de extração e transformação
PostgreSQL	14+	Armazenamento dos dados
Power BI Desktop	Atual	Visualização (opcional — pode usar CSV)

12.3. Bibliotecas Python

Biblioteca	Finalidade
requests	Chamadas HTTP à API
psycopg2 ou SQLAlchemy	Conexão com PostgreSQL
pandas	Manipulação de dados (opcional, conveniência)
logging	Registro de execução
schedule ou agendador do SO	Execução diária

13. Cronograma estimado

Etapa	Descrição	Duração estimada
1	Criação do banco (DDL, schemas, tabelas)	1 dia
2	Script de extração com paginação	1 dia
3	Script de detalhamento (loop por ID: detalhes + autores + temas + tramitações)	2 dias
4	Views analíticas + matching de keywords	1 dia
5	Carga incremental diária	1 dia
6	Dashboard Power BI ou export CSV	1 dia
7	Testes e validação com o time político	2 dias

Etapa	Descrição	Duração estimada
Total estimado		~9 dias úteis

14. Evolução futura

Funcionalidades previstas para fases posteriores ao MVP, conforme demanda:

Fase 2 — Enriquecimento: inclusão de pautas de comissões (/eventos, /orgaos/{id}/pauta), votações (/votacoes) e dados de relatores. Permite saber não apenas onde a proposição está, mas quando será votada e como cada deputado votou.

Fase 3 — Alertas: geração automática de relatório semanal (e-mail ou arquivo compartilhado) com proposições novas, proposições que movimentaram e proposições pautadas para votação.

Fase 4 — Inteligência: scoring de relevância ponderado por keywords, classificação assistida por LLM para ementas ambíguas, inclusão de dados do Senado Federal.

Migração para nuvem: caso o projeto prove valor e o acesso precise ser compartilhado com mais pessoas ou integrado ao portal ABCS, a migração para a infraestrutura AWS existente (S3 + Glue + PostgreSQL) pode ser feita sem alteração na lógica de negócio — apenas redirecionando a conexão do banco.

15. Referências

- API de Dados Abertos da Câmara dos Deputados:
<https://dadosabertos.camara.leg.br/swagger/api.html>
- Relatório Técnico v0.1 — Regras de Extração e Tratamento (documento interno)
- Relatório Técnico — Levantamento Inicial de Temas (documento interno)